문서 처리 파이프라인 (실습)

2-1. 문서 로더 (실습)
2-2. 청킹 전략 (실습)
2-3. 메타데이터 설계 (실습)

2-1. 문서 로더 (실습)

> 교육용 사내 정책 문서 3종을 만들고 Document 객체로 불러와봅니다.

RAG의 첫 단계는 파일에서 텍스트를 꺼내는 것입니다.

LangChain은 파일 형식마다 로더를 제공하며 로더는 모두 `Document` 객체의 리스트를 반환합니다. Document는 본문 문자열(`page_content`)과 부가 정보(`metadata`)로 구성됩니다.

형식이 달라도 결과는 같은 타입입니다. 그래서 뒤 단계인 청킹과 임베딩은 로더가 무엇이었는지 신경 쓰지 않아도 됩니다.

실무에서 자주 쓰는 문서 파싱 도구를 형식별로 정리하면 다음과 같습니다.

형식	도구	특징
PDF	pdfplumber (PDFPlumberLoader)	표 추출에 강하고 한글 PDF 호환성이 좋음
PDF	pypdf (PyPDFLoader)	가볍고 빠름, 표는 약함
Word	python-docx (Docx2txtLoader 등)	.docx 문단과 표 접근 가능
여러 형식 통합	unstructured (UnstructuredFileLoader)	PDF, PPT, HTML 등 거의 모든 형식, 무겁지만 만능
마크다운, 텍스트	TextLoader	가장 단순, 오늘 실습의 기본

오늘 실습은 마크다운 파일을 기본으로 합니다. PDF는 파일마다 파싱 품질 편차가 커서 파이프라인 학습에 방해가 되기 때문입니다.

PDF는 글자를 좌표에 찍어 놓은 그림에 가깝습니다. 그래서 폰트나 저장 방식에 따라 한글이 깨질 수 있습니다.

깨졌을 때는 pdfplumber를 먼저 시도합니다. 그래도 깨지면 원본을 마크다운이나 워드로 받는 것이 정답입니다. 스캔본 PDF는 텍스트가 아예 없으므로 OCR(광학 문자 인식)이 필요합니다.

오늘 사용할 데이터는 사내 정책 문서 3종(연차 규정, 재택근무 규정, 출장 규정)입니다. 실제 삼성SDS 규정과 무관한 교육용 더미 데이터입니다. 이 세 파일은 오늘 하루와 6일차 통합 실습까지 계속 씁니다.

따라하기

1. day2_practice 아래에 docs 폴더를 만듭니다.

```
# day2_practice 안에서 실행합니다
mkdir docs
```

2. 아래 내용을 `docs/leave_policy.md` 로 저장합니다.

```
# 연차 규정

본 문서는 교육용 더미 데이터입니다.

## 부여 기준
연차 휴가는 연 15일이 부여됩니다. 입사 1년 미만은 매월 1일씩 최대 11일까지 받을 수 있습니다.
3년 이상 근속자는 매 2년마다 1일이 더해져 최대 25일까지 받을 수 있습니다.

## 이월
미사용 연차는 5일까지 다음 해로 이월할 수 있습니다. 이월한 연차는 다음 해 6월 말까지 먼저 사용해야 합니다.

## 신청 절차
연차는 사용 5영업일 전까지 인사 포털에서 연차 신청서(양식 HR-103)로 신청합니다.
부서장 승인 후 확정되며 긴급한 경우 사후 신청은 다음 영업일 오전까지 마쳐야 합니다.
```

3. 아래 내용을 `docs/remote_work_policy.md` 로 저장합니다.

```
# 재택근무 규정

본 문서는 교육용 더미 데이터입니다.

## 운영 기준
재택근무는 주 2일까지 사용할 수 있습니다. 팀 필수 출근일(수요일)에는 재택근무를 사용할 수 없습니다.
신청은 전주 목요일까지 인사 포털에서 재택근무 신청서(양식 RW-201)로 제출합니다.

## 보안 수칙
재택근무 중 사내망 접속은 VPN 사용이 필수입니다.
개인 PC를 사용할 경우 보안 점검 프로그램을 설치하고 보안 서약서를 제출해야 합니다.
사내 자료는 승인 없이 개인 저장소나 외부 서비스에 올릴 수 없습니다.

## 근태
재택근무 시작과 종료 시 협업 메신저에 상태를 등록합니다. 근무 시간은 사무실 근무와 동일하게 적용됩니다.
```

4. 아래 내용을 `docs/travel_policy.md` 로 저장합니다.

```
# 출장 규정

본 문서는 교육용 더미 데이터입니다.

## 신청과 승인
출장은 출발 3영업일 전까지 인사 포털에서 출장 신청서(양식 TR-102)로 신청하고 부서장 승인을 받아야 합니다.
항공권과 숙박은 출발 전에 사내 출장 시스템에서 미리 예약해야 합니다.

## 경비 기준
국내 출장 식비 한도는 하루 5만원, 해외 출장 식비 한도는 하루 8만원입니다.
숙박비는 국내 1박 12만원, 해외 1박 20만원 한도입니다.

## 정산
출장비는 복귀 후 7영업일 안에 영수증을 첨부해 정산해야 합니다.
법인카드 사용 내역은 매월 말일까지 경비 시스템에 등록합니다.
```

5. 아래 코드를 `loaders.py` 로 저장하고 실행해 파일 하나와 폴더 전체를 각각 불러옵니다.

```
# loaders.py - 문서 로더
from langchain_community.document_loaders import TextLoader, DirectoryLoader

# 1) 파일 하나 로드
loader = TextLoader("docs/leave_policy.md", encoding="utf-8")
docs = loader.load()
print(f"문서 수: {len(docs)}")
print(f"metadata: {docs[0].metadata}")
print(f"본문 앞 80자: {docs[0].page_content[:80]}")

# 2) 폴더 통째로 로드
dir_loader = DirectoryLoader(
    "docs",
    glob="**/*.md",
    loader_cls=TextLoader,
    loader_kwargs={"encoding": "utf-8"},
)
all_docs = dir_loader.load()
print(f"\n전체 문서 수: {len(all_docs)}")
for d in all_docs:
    print(f"  - {d.metadata['source']} ({len(d.page_content)}자)")
```

막힐 때

- `UnicodeDecodeError: 'cp949' codec can't decode...`

  Windows에서 가장 흔한 오류입니다. TextLoader의 기본 인코딩이 OS 기본값을 따르기 때문입니다. 항상 `encoding="utf-8"` 을 명시합니다. 파일 자체가 다른 인코딩으로 저장된 경우라면 `autodetect_encoding=True` 옵션을 씁니다.

- 전체 문서 수가 3이 아닙니다.

  docs 폴더에 다른 파일이 섞였거나 파일 확장자가 다른 경우입니다. `glob="**/*.md"` 처럼 확장자를 지정하는 습관을 들입니다.

확인: 전체 문서 수 3과 파일별 source와 글자 수가 출력되면 성공입니다.

> 어떤 형식이든 로더를 거치면 같은 Document 타입이 된다는 걸 확인했습니다.

2-2. 청킹 전략 (실습)

> chunk_size와 overlap을 바꿔가며 분할 결과가 어떻게 달라지는지 비교해봅니다.

문서를 통째로 검색 단위로 쓰면 두 가지 문제가 생깁니다. 먼저 한 문서에 여러 주제가 섞여 있으면 벡터가 평균적인 의미가 되어 어떤 질문과도 어중간하게 비슷해집니다. 또 통째로 LLM에 넣으면 질문과 무관한 부분까지 함께 들어가 답변 품질이 떨어집니다. 그래서 문서를 작은 조각(청크)으로 나눕니다.

기본 도구는 `RecursiveCharacterTextSplitter` 입니다. Recursive라는 이름은 단락, 줄바꿈, 공백, 글자 순서로 경계를 시도하기 때문에 붙었습니다. 문장이 가급적 중간에서 잘리지 않습니다. 조정값은 두 가지입니다.

- `chunk_size` : 청크 하나의 최대 글자 수입니다. 작으면 정밀하지만 맥락이 부족하고 크면 맥락은 풍부하지만 검색이 흐려집니다.
- `chunk_overlap` : 이웃 청크끼리 겹치는 글자 수입니다. 경계에 걸린 정보가 잘려나가는 것을 막습니다.

따라하기

1. 아래 코드를 `chunking.py` 로 저장하고 세 가지 설정으로 같은 문서를 나눠 결과를 비교합니다.

```
# chunking.py — 청킹 설정 비교
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

docs = TextLoader("docs/leave_policy.md", encoding="utf-8").load()

for size, overlap in [(100, 0), (100, 30), (300, 50)]:
    splitter = RecursiveCharacterTextSplitter(chunk_size=size, chunk_overlap
=overlap)
    chunks = splitter.split_documents(docs)
    print(f"\n=== chunk_size={size}, overlap={overlap} -> {len(chunks)}개 청크
===")
    for i, c in enumerate(chunks):
        print(f"[{i}] ({len(c.page_content)}자) {c.page_content[:40].replace
(chr(10), ' ')}...")
```

   `chr(10)` 은 줄바꿈 문자를 뜻합니다. 청크 안의 줄바꿈을 공백으로 바꿔 한 줄로 찍으려고 쓴 것이라 청킹 자체와는 상관없습니다.

2. 출력된 세 설정을 비교합니다. 같은 문서인데 청크 개수와 경계가 달라집니다.
    - `chunk_size=100, overlap=0` : 청크가 잘게 나뉘고 경계에 걸린 문장이 앞뒤로 흩어집니다. 경계 정보가 사라져 실무에서는 거의 쓰지 않습니다.
    - `chunk_size=100, overlap=30` : 경계에 걸린 내용이 양쪽 청크에 모두 남습니다.
    - `chunk_size=300, overlap=50` : 청크당 맥락은 풍부하지만 부여 기준과 이월이 한 청크에 섞입니다. 오늘 실습의 기본값입니다.
3. 마크다운 문서라면 헤더 구조를 살리는 분할기가 더 좋습니다. 아래 코드를 `chunking_md.py` 로 저장하고 실행합니다.

```
# chunking_md.py — 마크다운 헤더 기준 분할
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import MarkdownHeaderTextSplitter

docs = TextLoader("docs/leave_policy.md", encoding="utf-8").load()

headers = [("#", "title"), ("##", "section")]
md_splitter = MarkdownHeaderTextSplitter(headers_to_split_on=headers)
md_chunks = md_splitter.split_text(docs[0].page_content)
for c in md_chunks:
    print(c.metadata, "->", c.page_content[:30])
```

헤더 정보가 metadata로 자동으로 붙습니다. 검색된 청크가 어느 섹션 소속인지 알 수 있어 출처 표시에 유용합니다.

실무 출발점은 한국어 기준 chunk_size 500-1000자와 overlap 50-150자입니다. 오늘 실습 문서는 짧아서 작은 값을 썼습니다.

막힐 때

- 한국어 문서인데 토큰이 예상보다 많이 나갑니다.

  영어는 대략 4자당 1토큰이지만 한국어는 1-2자당 1토큰입니다. 같은 chunk_size라도 한국어 청크가 토큰을 2-3배 더 씁니다. 컨텍스트 한도가 빠듯하면 `TokenTextSplitter` 로 토큰 기준 분할을 검토합니다.

- 문장 중간에서 자꾸 잘립니다.

  한국어는 마침표 없이 개행으로 문장을 구분하는 문서가 많습니다. `separators=["\n\n", "\n", "다. ", ". ", " ", ""]` 처럼 한국어 종결 패턴을 추가하면 좋아집니다.

- 표가 있는 문서가 이상하게 잘립니다.

  표 한가운데를 자르면 행과 헤더가 분리되어 의미가 사라집니다. 표가 많은 문서는 chunk_size를 키우거나 표 단위 수동 분할을 고려합니다.

확인: 같은 문서가 설정에 따라 다른 개수의 청크로 나뉘고 overlap을 준 설정에서 이웃 청크의 앞뒤가 겹치는 것을 보면 성공입니다.

> 청크 크기와 겹침이 검색 정밀도와 맥락 사이의 트레이드오프라는 걸 확인했습니다.

2-3. 메타데이터 설계 (실습)

> 출처, 부서, 개정일 메타데이터를 청크에 부착해봅니다.

메타데이터는 청크에 붙는 꼬리표입니다. 세 가지 목적으로 씁니다.

1. 출처 표시: 답변에 "`leave_policy.md` 에 따르면"을 붙이려면 청크가 어느 파일 출신인지 알아야 합니다.
2. 필터 검색: "인사팀 문서에서만 찾아줘" 같은 조건 검색이 가능해집니다.
3. 최신성 관리: 문서 개정일을 기록해 두면 낡은 규정과 새 규정을 구분할 수 있습니다.

로더가 `source` 는 자동으로 넣어 주지만 부서나 날짜 같은 업무 정보는 직접 설계해서 붙여야 합니다. 이번 과정의 메타데이터 키는 `source` , `department` , `updated_at` 세 개로 고정합니다. 4일차 구조화 데이터와 6일차 종합 시나리오에서도 같은 키 체계를 씁니다.

따라하기

1. 아래 코드를 `metadata.py` 로 저장합니다. 파일명은 그대로 써야 합니다. 3번째 세션 3-3의 벡터 저장소와 5번째 세션 5-2의 하이브리드 검색이 `from metadata import load_chunks` 로 이 파일을 import하기 때문입니다.

```
# metadata.py — 메타데이터 부착 (뒤 실습에서 재사용)
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 파일별 업무 메타데이터 (교육용 더미 데이터)
FILE_META = {
    "docs/leave_policy.md":       {"department": "인사팀", "updated_at": "2026
-01-15"},
    "docs/remote_work_policy.md": {"department": "인사팀", "updated_at": "2026
-03-02"},
    "docs/travel_policy.md":      {"department": "총무팀", "updated_at": "2025
-11-20"},
}


def load_chunks():
    """세 문서를 로드해 메타데이터를 붙이고 청크로 나눕니다."""
    splitter = RecursiveCharacterTextSplitter(chunk_size=300, chunk_overlap=
50)
    all_chunks = []
    for path, meta in FILE_META.items():
        docs = TextLoader(path, encoding="utf-8").load()
        for d in docs:
            d.metadata.update(meta)      # source 위에 업무 정보 추가
        all_chunks.extend(splitter.split_documents(docs))
    return all_chunks


if __name__ == "__main__":
    chunks = load_chunks()
    for c in chunks[:3]:
        print(c.metadata)
    print(f"\n총 청크 수: {len(chunks)}")
```

2. 출력된 metadata에 `source` , `department` , `updated_at` 세 키가 모두 있는지 확인합니다.

   맨 아래 `if __name__ == "__main__":` 블록은 **이 파일을 직접 실행했을 때만** 도는 부분입니다. 다른 파일에서 `from metadata import load_chunks` 로 가져다 쓸 때는 실행되지 않습니다.
   파이썬은 import하는 순간 그 파일을 위에서부터 한 번 실행하기 때문에, 이 가드가 없으면 import만 해도 테스트 코드가 같이 돕니다. Java로 치면 클래스마다 둔 `main` 메서드에 가깝습니다.

`FILE_META` 는 파일 경로를 키로, 부서와 개정일을 값으로 담은 딕셔너리입니다. `.items()` 로 돌리면 키와 값을 한 번에 받아 쓸 수 있습니다. 자바의 `Map.entrySet()` 순회와 같습니다.

문서에 metadata를 붙인 뒤 split하면 모든 청크가 그 metadata를 그대로 물려받습니다. 그래서 메타데이터는 청킹 전에 붙입니다.

막힐 때

- 메타데이터에 리스트를 넣었더니 저장할 때 오류가 납니다.

  Chroma는 metadata 값으로 str, int, float, bool만 받습니다. 태그 여러 개가 필요하면 `"tags": "인사,휴가,복지"` 처럼 문자열로 합쳐 넣습니다.

- 날짜로 범위 검색을 하고 싶습니다.

  문자열 "2026-01-15"는 정렬은 되지만 범위 비교 필터가 불편합니다. 날짜 필터가 필요하면 정수 타임스탬프를 함께 넣는 것이 실무 요령입니다.

- Windows에서 source가 `docs\leave_policy.md` 로 나옵니다.

  경로 구분자 차이입니다. 비교 코드를 쓸 때는 `pathlib.Path(path).as_posix()` 로 통일합니다.

확인: 모든 청크의 metadata에 source, department, updated_at 세 키가 보이면 성공입니다.

> 청킹 전에 붙인 메타데이터를 모든 청크가 상속받는다는 걸 확인했습니다.
